

Deep Dive — Agentic Security, Data Risks, Red-Teaming, and the Supply Chain

Part 13 —  AI Security · 13-ai-security/notes/02-agentic-security-and-red-teaming.md

This note picks up where notes/01 [13-ai-security/notes/01-owasp-and-prompt-injection.md] left off: prompt injection and jailbreaking are the mechanism, this note is where the mechanism meets real systems — agents with tools, vector stores full of your company's documents, models trained on data you didn't fully vet, and a production request path that either has guardrails in it or doesn't.

1. Excessive agency: the highest-stakes OWASP category in practice

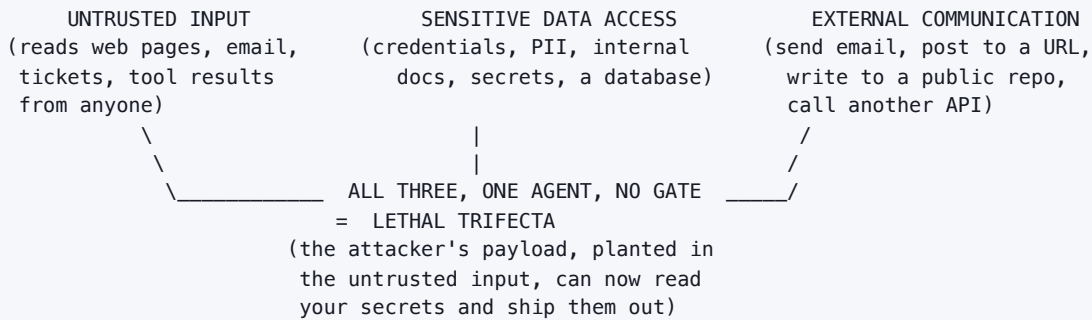
Excessive agency (LLM03) is what happens when a system hands a model more capability than the task in front of it needs, "just in case," or because one general-purpose tool was easier to build than three narrow ones. Three components compound it, and OWASP's own framing separates them cleanly:

- **Excessive functionality** — the tool can do more than the task requires. A "read calendar" tool that can also delete events. A "look up order" tool built on top of a database client that can run arbitrary SQL because that was the fastest way to ship it.
- **Excessive permissions** — the tool's *credentials* are broader than the tool's *purpose*. A tool that only needs to read one table connects with a service account that can write to all of them.
- **Excessive autonomy** — the system lets the model take consequential action without a human checkpoint, because building the checkpoint felt like it would slow down the "wow, it's agentic" demo.

The fix for each is the same instinct as classical least-privilege, applied to a system whose "user" is a model that can be talked into wanting things it shouldn't: **scope the tool to the task, scope the credential to the tool, and gate autonomy at the point where an action becomes hard to undo**. Module 06's agent notes [06-ai-agents] build the tool-calling loop this constrains; treat every tool you add there as a capability grant that needs its own justification, not a free add.

2. The lethal trifecta, formalized

Three properties, and the danger is in their **conjunction**, not any one alone:



Any one leg is fine alone. A research agent that reads the open web is fine — it has nothing sensitive to lose and nowhere consequential to send it. A tool that reads a customer's own account balance and displays it back to that same customer is fine — no untrusted third-party content in the loop, and the "communication" is just rendering to the user who already has legitimate access to that data. **The danger only appears when a single reachable execution context has all three**, because that is exactly the shape of "attacker plants an instruction in content the agent will read, the agent already holds data worth stealing, and the agent has a way to ship it out."

A decision framework for spotting it in a diagram

Walk every node (agent, subgraph, or single LLM call with a bound tool list) in your architecture and ask three yes/no questions:

1. **Does this node ingest content from a source not fully controlled by you or your organization?** (web pages, uploaded files, customer email, third-party API responses, MCP tool results from a server you don't operate) → untrusted input, yes/no.
2. **Does this node have a live credential, connection, or context that reaches confidential data?** (API keys, a database connection, a secrets file, an internal knowledge base with non-public content) → sensitive data access, yes/no.
3. **Does this node have a tool that causes an effect outside the current conversation** — sends a message, posts to a URL, writes to storage another system reads, calls another service — **that a human does not have to approve first?** → ungated external communication, yes/no.

If all three answers are "yes" for the *same node*, you have found a lethal-trifecta configuration.

`code/lethal_trifecta_analyzer.py` [13-ai-security/code/lethal_trifecta_analyzer.py] turns exactly this checklist into a small static analyzer you can run over a declarative description of an agent's tools — run it before you trust your own diagram-reading.

Fixing it: three moves, in order of preference

1. **Split into separate agents at the trust boundary.** One agent reads untrusted content and produces a narrow, structured, schema-constrained output (a summary, an extracted field) with *no* sensitive data access and *no* communication tool. A second agent consumes that structured output — never the raw untrusted text — and holds the sensitive access or the communication capability. The injected instruction has nowhere to travel, because the handoff between the two agents is data, not free text the second agent re-reads as instructions.

2. **Add a human approval gate on the third leg.** If splitting isn't practical, put the communication or write action behind an explicit approval step — Module 16's LangGraph `interrupt()` [16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md] is the concrete mechanism; Module 12 [12-ai-governance] is why an auditor wants to see it. This converts "the model did something" into "a named person authorized this specific action."
3. **Remove a capability.** Sometimes the honest fix is that the agent doesn't need all three. Does the research assistant really need to *post* to Slack, or would returning a draft for the user to send satisfy the actual requirement? Removing agency is not a downgrade; it's usually the discovery that the task never needed it.

Tool-permission design and MCP as a trust boundary

Every tool you bind to a model is a capability grant — design it with the same rigor as an IAM policy: least privilege per tool, scoped credentials per tool (not one shared service account for the whole agent), and an explicit allowlist of which tools are reachable in which context (this is exactly the "privilege separation" defense from notes/01, §5.2 [13-ai-security/notes/01-owasp-and-prompt-injection.md]). Module 09's MCP security model [09-mcp] treats an MCP server as a trust boundary for exactly this reason: a tool exposed by a third-party MCP server is code you did not write, running with whatever permissions you granted the connection, and its results land back in your model's context as data your model will read and may act on. Vet an MCP server the way you'd vet a new production dependency — read what it can do, not just what its README says it's for — before wiring it into an agent that also holds sensitive data.

3. Model-level attacks

These are lower-frequency in most application-layer incident reports than prompt injection, but they are real, and you should recognize them by name.

- **Adversarial examples** — small, often imperceptible input perturbations that flip a model's output (a few pixels that make an image classifier see a different class; this is classical ML security, covered mechanically in Module 01 [01-ml-foundations] and Module 02 [02-deep-learning]). For LLMs the analogue is adversarial *suffixes* — token sequences, sometimes optimized by gradient search against an open-weight model, that reliably suppress a refusal when appended to a harmful request. The defensive takeaway is the same as jailbreaking generally: test with adversarial suffixes from public research as part of your eval suite, don't assume instruction-tuning alone is robust to optimization-based attacks.
- **Model extraction / theft via API querying** — an attacker repeatedly queries your model's public API and trains a "student" model to imitate its input-output behavior, effectively stealing the economic value of your training investment without touching the weights. Module 04's papers [04-llm/papers/PAPERS.md] on imitating proprietary LLMs (Gudibande et al.) show the flip side — imitation captures *style*, not the underlying capability — which is some comfort but not a defense. Mitigations: rate limiting per API key (this is also an Unbounded Consumption / LLM06 control), watermarking or fingerprinting outputs, query pattern anomaly detection (a client issuing millions of diverse, systematically-varied prompts looks nothing like a real user), and contractual terms that make extraction a breach you can act on.

- **Membership inference** — determining whether a specific record was part of a model's training set, by exploiting the fact that models tend to be more confident on data they've seen. For an LLM this matters most as a *privacy* attack against training data: if you fine-tuned on customer support transcripts, an attacker who can query the model and observe confidence or exact-match behavior may be able to establish that a specific customer's data was in the training set — a disclosure in itself, independent of the model ever emitting the raw text. Mitigation: differential privacy during training (adds calibrated noise, at a cost to accuracy), aggressive deduplication, and simply not fine-tuning on data you aren't prepared to have its membership be inferable.
-

4. Data security specific to AI systems

Training data poisoning and backdoors

Poisoning corrupts a small fraction of the pretraining, fine-tuning, or RLHF preference data so the resulting model has a hidden trigger behavior — normal performance on everything, until a specific trigger phrase or pattern appears, at which point it does something the attacker chose (leaks data, inserts a vulnerability into generated code, flips a classification). The attack is conceptually simple and the defense is unglamorous and mostly the same as classical data-pipeline hygiene, scaled up: provenance tracking for every data source that enters a training run, deduplication and outlier detection (a backdoor trigger is usually a rare pattern, which makes it statistically detectable if you're looking), decontamination against known-bad sources, and — for anything fine-tuned on externally-sourced or crowdsourced data — treating that dataset with the same supply-chain suspicion you'd apply to a third-party code dependency (§6).

PII leakage through model outputs (memorization)

Large models memorize verbatim spans of low-frequency training data — a phone number that appeared once in a scraped forum post, a full name paired with an address in a leaked breach dataset that ended up in a web crawl. A sufficiently targeted prompt can sometimes extract that exact span back out, which is a disclosure incident even though "the model wasn't hacked" in any classical sense. Mitigation: PII scrubbing and deduplication *before* training (the highest-leverage point — you cannot memorize what was never in the corpus), differential privacy for sensitive fine-tuning corpora, and output-side PII detection as a last-resort net (§5).

PII leakage through RAG (retrieving and echoing sensitive documents)

A distinct failure mode from memorization, and arguably more common in production: the model never memorized anything, but your retrieval pipeline fetched a document the *current user* was never authorized to see, and the model — doing exactly what it's supposed to do — faithfully summarized it back to them. This is LLM09 (Vector and Embedding Weaknesses) in its most common real form. The root cause is almost always that the vector store's access control lags the source system's: a document gets deleted or its permissions get tightened in the source system, but the embedding that was computed from it months ago is still sitting in the index, fully retrievable. **The fix is not a smarter prompt; it's propagating the source system's ACLs into the retrieval filter, at query time, before the top-k documents are even selected** — filter by the querying user's permissions as a `WHERE` clause on the vector search, not as an instruction to the model asking it to

"please only use documents the user can see." See Module 08's RAG [08-rag] for the retrieval architecture this constrains.

Embedding inversion

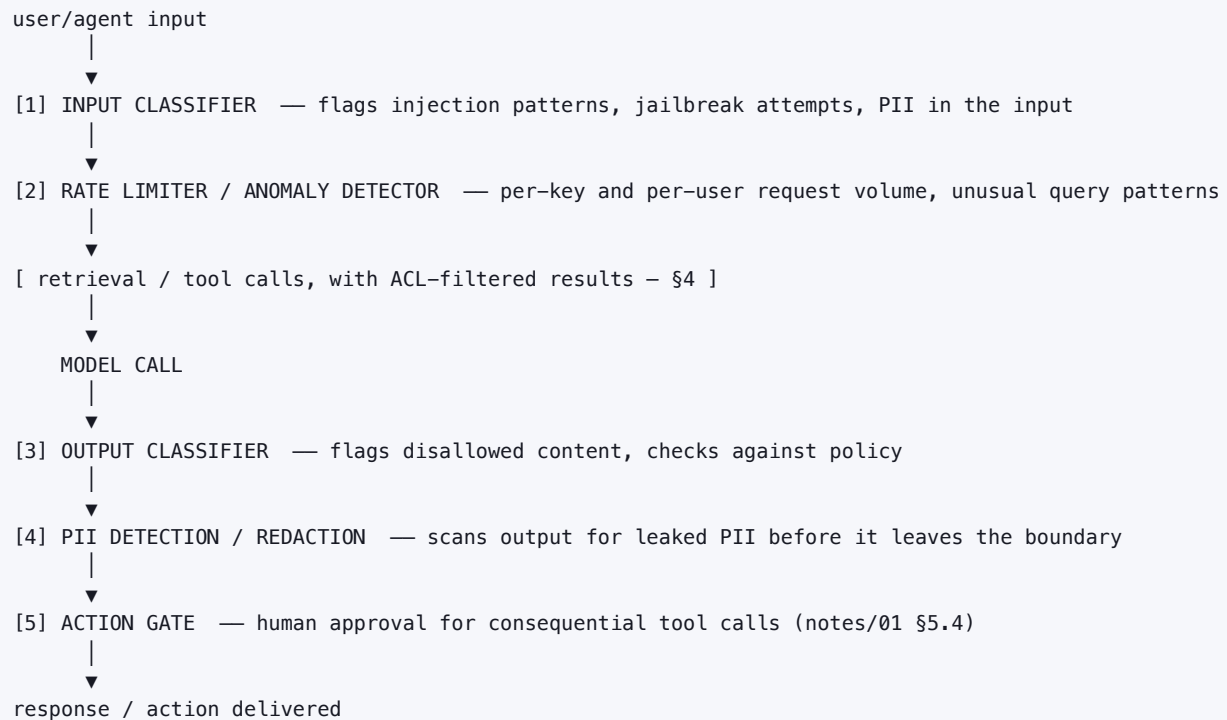
Embeddings are usually treated as opaque numeric vectors, but they are not one-way in the cryptographic sense: research has repeatedly shown that a vector, or even just its nearest neighbors, can be inverted back into text that closely approximates the original content — enough to recover names, identifiers, or sentence-level meaning from what looked like "just numbers." Practical consequence: **a vector database is a data store containing recoverable information about your source documents, and needs the same access control, encryption-at-rest, and retention policy as the documents themselves** — not the lighter treatment teams often give it because "it's just embeddings."

Data governance for what goes into a vector store

A short, concrete checklist, because this is where LLM09 becomes an operational discipline rather than a paragraph: classify source documents *before* embedding (not after — retrofitting classification onto an existing index is a project, not a config change); propagate source ACLs into retrieval filters, always; set a re-index or TTL policy so deleted or permission-changed source documents actually leave the index; log what was retrieved for what query, so an incident is investigable after the fact; and treat embedding models themselves as a supply-chain dependency (§6) — a third-party embedding API sees every document you send it.

5. Guardrails and monitoring in production

Guardrails are the runtime enforcement of everything above — the layer that has to work even when a developer forgot to, or a new attack pattern nobody wrote a rule for shows up first in traffic rather than in a test suite. Place them explicitly in the request path; Module 10's reference architecture [10-ai-architecture] is where the full request-flow diagram lives — this is the security-relevant subset of it:



Two honest caveats worth stating out loud: **[1] and [3] are classifiers, and classifiers have false negatives against novel or obfuscated attacks** — this is the same limitation named in notes/01's delimiting discussion, now at the infrastructure layer. And **rate limiting and anomaly detection ([2]) catch volume-shaped abuse (extraction, denial-of-wallet) but do nothing against a single, well-crafted injected instruction in one request** — different layer, different threat. No guardrail here replaces the architectural controls in §2; guardrails are the safety net under an architecture that should already be sound, not a substitute for one that isn't.

6. Red-teaming AI systems as a practice

A structured LLM red-team engagement is not a penetration test with a different target — it differs in what "found a vulnerability" even means.

	Traditional pentest	LLM red-team
Target	Code, network, infrastructure	Model behavior <i>and</i> the system wrapped around it
"Vulnerability"	A reproducible exploit with a CVE-shaped root cause	Often a probabilistic behavior — works 30% of the time, or only in certain phrasings
Reproducibility	Deterministic, given the same input	Frequently non-deterministic; sampling and model updates change results run to run
Scoring	Binary: exploitable or not	A graded severity + success-rate scale — one jailbreak attempt succeeding once is a different finding than succeeding reliably
Tooling	Network scanners, fuzzers, known-CVE databases	Adversarial prompt libraries, automated attack generators, human red-teamers improvising conversationally
Coverage model	Enumerate known vulnerability classes	Enumerate known <i>attack techniques</i> (§6 of notes/01) and try each against every capability the system exposes

What a real engagement actually produces: an **adversarial prompt library** (curated known-injection and known-jailbreak templates, continuously updated as new techniques publish), a mix of **automated red-teaming** (another model, prompted or fine-tuned to generate and iterate attacks at scale — cheap, wide coverage, weaker at genuinely novel techniques) and **human red-teaming** (creative, multi-turn, good at exactly the crescendo-style escalation automated tooling tends to miss), and a **scored report**: each attempt logged with the technique used, the target capability, whether it succeeded, at what severity, and — critically — whether it's a single-turn or multi-turn finding, because those require different fixes. The deliverable a red-team engagement should hand you not just a list of failures but a prioritized list mapped back to the OWASP categories in notes/01, so the fix work has an owner and a category, not just a pile of transcripts.

7. Supply chain security for AI

LLM04 (Supply Chain) extends classical software supply-chain security — the discipline behind SBOMs (software bills of materials), dependency pinning, and provenance attestation — to artifacts classical tooling was never built to inspect:

- A **third-party model weight file** is an opaque binary blob you are choosing to execute. Unlike a pinned npm package, you generally cannot diff it against source, and a backdoored fine-tune (§4) can be behaviorally invisible until triggered. Mitigate with provenance: prefer weights from a source that publishes training data lineage and a model card, verify checksums against the publisher's signed release, and scan the surrounding artifact (a pickle-format checkpoint can itself contain arbitrary code — a genuinely known attack vector, which is why `safetensors` exists as a non-executable serialization format).
- A **compromised MCP server** (§2, Module 09 [09-mcp]) is the closest AI-native analogue to a compromised npm package: you install it for one advertised capability, and it runs with whatever access you granted, able to return results your model treats as trustworthy data.
- A **poisoned fine-tuning dataset**, especially one sourced from a public or crowdsourced repository, is a supply-chain input the same way a dependency's transitive package is — vet its provenance, spot-check samples, and run poisoning-detection heuristics (§4) before training on it.

- **A compromised dependency in your ML pipeline** — the training or serving stack itself (a compromised `pip` package that exfiltrates data during training, a poisoned base Docker image) is the exact same classical supply-chain risk that non-AI software has faced for years, and the same controls apply: SBOM generation for your training and serving environments, pinned and hash-verified dependencies, and signed, reproducible builds. **The AI-specific addition is that the "dependency" now includes model weights and datasets, which need the same provenance discipline as code and currently, industry-wide, usually don't get it.** Module 14's compliance material [14-ai-compliance] covers the emerging regulatory expectation (an "AI BOM") that formalizes exactly this gap.

8. Forward and backward links

Idea here	Where it returns
Lethal trifecta mechanism	notes/01, §5.2 [13-ai-security/notes/01-owasp-and-prompt-injection.md] and Module 16 / Claude Code [16-ai-tech-stack/tracks/claude-code/notes/01-claude-code-core-concepts.md]
Tool permission design	Module 06 — AI Agents [06-ai-agents]
MCP as a trust boundary	Module 09 — MCP [09-mcp]
Approval gates	Module 16 / LangGraph [16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md] and Module 12 — Governance [12-ai-governance]
RAG ACL propagation	Module 08 — RAG [08-rag]
Guardrail placement in the request path	Module 10 — AI Architecture [10-ai-architecture]
Adversarial examples, mechanistically	Module 01 [01-ml-foundations], Module 02 [02-deep-learning]
AI BOM / regulatory provenance	Module 14 — AI Compliance [14-ai-compliance]
Red-team findings as an eval suite	Module 15 — AI Evals [15-ai-evals]